Scenario

In this scenario, you're a security analyst who must monitor traffic on your employer's network. You'll be required to configure Suricata and use it to trigger alerts.

- First, you'll explore custom rules in Suricata. Second, you'll run.
- Suricata with a custom rule in order to trigger it, and examine the output logs in the fast.log file.
- Finally, you'll examine the additional output that Suricata generates in the standard eve.json log file.

For the purposes of the tests you'll run in this lab activity, you've been supplied with a sample.pcap file and a custom.rules file. These reside in your home folder.

Let's define the files you'll be working with in this lab activity:

The sample.pcap file is a packet capture file that contains an example of network traffic data, which you'll use to test the Suricata rules. This will allow you to simulate and repeat the exercise of monitoring network traffic.

The custom.rules file contains a custom rule when the lab activity starts. You'll add rules to this file and run them against the network traffic data in the sample.pcap file.

The fast.log file will contain the alerts that Suricata generates. The fast.log file is empty when the lab starts. Each time you test a rule, or set of rules, against the sample network traffic data, Suricata adds a new alert line to the fast.log file when all the conditions in any of the rules are met. The fast.log file can be located in the /var/log/suricata directory after Suricata runs.The fast.log file is considered to be a depreciated format and is not recommended for incident response or threat hunting tasks but can be used to perform quick checks or tasks related to quality assurance.

The eve.json file is the main, standard, and default log for events generated by Suricata. It contains detailed information about alerts triggered, as well as other network telemetry events, in JSON format. The eve.json file is generated when Suricate runs, and can also be located in the /var/log/suricata directory.

Task 1. Examine a custom rule in Suricata

The /home/analyst directory contains a custom.rules file that defines the network traffic rules, which Suricata captures.

In this task, you'll explore the composition of the Suricata rule defined in the custom.rules file.

```
analyst@52391e51c5c1:~$ cat custom.rules
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"GET on wire"; flow:est
ablished,to_server; content:"GET"; http_method; sid:12345; rev:3;)
analyst@52391e51c5c1:~$
```

Here I used the cat command to display the rule in the custom.rules file.

The action is the first part of the signature. It determines the action to take if all conditions are met.

Actions differ across network intrusion detection system (NIDS) rule languages, but some common actions are alert, drop, pass, and reject.

Using our example, the file contains a single alert as the action. The alert keyword instructs to alert on selected network traffic. The IDS will inspect the traffic packets and send out an alert in case it matches.

Note that the drop action also generates an alert, but it drops the traffic. A drop action only occurs when Suricata runs in IPS mode.

The pass action allows the traffic to pass through the network interface. The pass rule can be used to override other rules. An exception to a drop rule can be made with a pass rule. For example, the following rule has an identical signature to the previous example, except that it singles out a specific IP address to allow only traffic from that address to pass.

The reject action does not allow the traffic to pass. Instead, a TCP reset packet will be sent, and Suricata will drop the matching packet. A TCP reset packet tells computers to stop sending messages to each other.

```
alert http $HOME_NET any -> $EXTERNAL_NET
any (msg:"GET on wire";
flow:established,to_server; content:"GET";
http_method; sid:12345; rev:3;)
```

The next part of the signature is the header. The header defines the signature's network traffic, which includes attributes such as protocols, source and destination IP addresses, source and destination ports, and traffic direction.

The next field after the action keyword is the protocol field. In our example, the protocol is http, which determines that the rule applies only to HTTP traffic.

The parameters to the protocol http field are $HOME_NET any -> $EXTERNAL_NET any. The arrow indicates the direction of the traffic coming from the $HOME_NET and going to the destination IP address $EXTERNAL_NET.

$HOME_NET is a Suricata variable defined in /etc/suricata/suricata.yaml that you can use in your rule definitions as a placeholder for your local or home network to identify traffic that connects to or from systems within your organization.

```
alert http $HOME_NET any -> $EXTERNAL_NET
any (msg:"GET on wire";
flow:established,to_server; content:"GET";
http_method; sid:12345; rev:3;)
```

The many available rule options allow you to customize signatures with additional parameters. Configuring rule options helps narrow down network traffic so you can find exactly what you're looking for. As in our example, rule options are typically enclosed in a pair of parentheses and separated by semicolons.

Let's further examine the rule options in our example:

- The msg: option provides the alert text. In this case, the alert will print out the text “GET on wire”, which specifies why the alert was triggered.
- The flow:established,to_server option determines that packets from the client to the server should be matched. (In this instance, a server is defined as the device responding to the initial SYN packet with a SYN-ACK packet.)
- The content:"GET" option tells Suricata to look for the word GET in the content of the http.method portion of the packet.
- The sid:12345 (signature ID) option is a unique numerical value that identifies the rule.
- The rev:3 option indicates the signature's revision which is used to identify the signature's version. Here, the revision version is 3.

To summarize, this signature triggers an alert whenever Suricata observes the text GET as the HTTP method in an HTTP packet from the home network going to the external network.

Task 2. Trigger a custom rule in Suricata

Now that you are familiar with the composition of the custom Suricata rule, you must trigger this rule and examine the alert logs that Suricata generates.

```
analyst@52391e51c5c1:~$ sudo suricata -r sample.pcap -S custom.rules -k no
ne

i: suricata: This is Suricata version 7.0.10 RELEASE running in USER mode
i: threads: Threads created -> RX: 1 W: 1 FM: 1 FR: 1   Engine started.
i: suricata: Signal Received.  Stopping engine.
i: pcap: read 1 file, 200 packets, 54238 bytes
```

Here we use the command sudo suricata -r sample.pcap -S custom.rules -k none which starts the Suricata application and processes the sample.pcap file using the rules in the custom.rules file.

- The -r sample.pcap option specifies an input file to mimic network traffic. In this case, the sample.pcap file.
- The -S custom.rules option instructs Suricata to use the rules defined in the custom.rules file.
- The -k none option instructs Suricata to disable all checksum checks.

Task 3. Examine eve.json output

In this task, you must examine the additional output that Suricata generates in the eve.json file.

The eve.json file is the standard and main Suricata log file and contains a lot more data than the fast.log file. This data is stored in a JSON format, which makes it much more useful for analysis and processing by other applications.


```
analyst@52391e51c5c1:~$ cat /var/log/suricata/eve.json
{"timestamp":"2022-11-23T12:38:34.624866+0000","flow_id":695532928922451,"
pcap_cnt":70,"event_type":"alert","src_ip":"172.21.224.2","src_port":49652
,"dest_ip":"142.250.1.139","dest_port":80,"proto":"TCP","pkt_src":"wire/pc
ap","tx_id":0,"alert":{"action":"allowed","gid":1,"signature_id":12345,"re
v":3,"signature":"GET on wire","category":"","severity":3},"http":{"hostna
me":"opensource.google.com","url":"/","http_user_agent":"curl/7.74.0","htt
p_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","sta
tus":301,"redirect":"https://opensource.google/","length":223},"app_proto"
:"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient":
3,"bytes_toserver":357,"bytes_toclient":788,"start":"2022-11-23T12:38:34.6
20693+0000","src_ip":"172.21.224.2","dest_ip":"142.250.1.139","src_port":4
9652,"dest_port":80}}
{"timestamp":"2022-11-23T12:38:58.958203+0000","flow_id":726729619997893,"
pcap_cnt":151,"event_type":"alert","src_ip":"172.21.224.2","src_port":5849
4,"dest_ip":"142.250.1.102","dest_port":80,"proto":"TCP","pkt_src":"wire/p
cap","tx_id":0,"alert":{"action":"allowed","gid":1,"signature_id":12345,"r
ev":3,"signature":"GET on wire","category":"","severity":3},"http":{"hostn
ame":"opensource.google.com","url":"/","http_user_agent":"curl/7.74.0","ht
tp_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","st
atus":301,"redirect":"https://opensource.google/","length":223},"app_proto
":"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient"
:3,"bytes_toserver":357,"bytes_toclient":797,"start":"2022-11-23T12:38:58.
955636+0000","src_ip":"172.21.224.2","dest_ip":"142.250.1.102","src_port":
58494,"dest_port":80}}
```

Here we will use the cat command to display the entries in the eve.json file. The output returns the raw content of the file.now if we want to return the entries in an improved format we need to use jq

```
analyst@52391e51c5c1:~$ jq . /var/log/suricata/eve.json | less
{
  "timestamp": "2022-11-23T12:38:34.624866+0000",
  "flow_id": 695532928922451,
  "pcap_cnt": 70,
  "event_type": "alert",
  "src_ip": "172.21.224.2",
  "src_port": 49652,
  "dest_ip": "142.250.1.139",
  "dest_port": 80,
  "proto": "TCP",
  "pkt_src": "wire/pcap",
  "tx_id": 0,
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 12345,
    "rev": 3,
    "signature": "GET on wire",
    "category": "",
    "severity": 3
  },
  "http": {
    "hostname": "opensource.google.com",
    "url": "/",
    "http_user_agent": "curl/7.74.0",
    "http_content_type": "text/html",
    "http_method": "GET",
    "protocol": "HTTP/1.1",
    "status": 301,
    "redirect": "https://opensource.google/",
    "length": 223
  },
  "app_proto": "http",
  "direction": "to_server",
  "flow": {
    "pkts_toserver": 4,
    "pkts_toclient": 3,
    "bytes_toserver": 357,
    "bytes_toclient": 788,
```

In this picture we used the command jq . /var/log/suricata/eve.json | less to display the entries in an improved format. The jq command can also be used to extract specific event data from the eve.json file.

```
analyst@52391e51c5c1:~$ jq -c "[.timestamp,.flow_id,.alert.signature,.prot
o,.dest_ip]" /var/log/suricata/eve.json
["2022-11-23T12:38:34.624866+0000",695532928922451,"GET on wire","TCP","14
2.250.1.139"]
["2022-11-23T12:38:58.958203+0000",726729619997893,"GET on wire","TCP","14
2.250.1.102"]
```

Here we used the command jq -c "[.timestamp,.flow_id,.alert.signature,.proto,.dest_ip]" /var/log/suricata/eve.json to extract only the timestamp, .flow_id, .alert.signature, .prot, .dest_ip in the log.